lab2

功能总结

将原有的 sys_get_time 和 sys_trace 实现通过当前任务的页表完成地址翻译，并根据 PTE 的 V、U、R、W 等标志检查访问权限；同时，还完成了 mmap 和 munmap，利用 MemorySet、MapArea、PageTable 与 FrameTracker 协同完成用户虚拟内存区域的创建、权限设置、物理页分配、映射解除与资源回收，最终通过测试。

问答题

问题1：

sv39页表的页表项组成包括0-9位是权限位，包括页表是否有效，是否可读，是否可写等等，剩余的44位是物理页号，分别是PPN[0]，PPN[1]，PPN[2]。

问题2

2.1 哪些异常可能由缺页导致？

指令页异常，读/写页异常

2.2 缺页发生时，相关重要寄存器中保存什么？

寄存器包括scause（记录异常的原因），sstatus（陷入前的特权级），sepc（当前的指令地址，用于处理完异常后恢复），steval（异常的一些信息，比如引起异常的虚拟地址），satp（保存当前使用的地址翻译模型和跟页表）

2.3 Lazy 策略有什么好处？

减少设置了但未真正使用到的页面造成物理内存的浪费，降低启动时的内存读取开销。

2.4 映射 10 GiB 连续虚拟内存大约需要多少页表内存？

以4KB作为一页的话，一共有2621440个页面，即一共需要映射2641440个物理页。最低级页表，每个页表项占8个字节，所以每一页有512个pte，所以最低页可以映射2Mb，所以最低页表需要5120张，占用20Mb；中间级页表，每张页表也有512个pte，每一个pte可以映射2Mb的最低级页表，所以一张

中级页表可以覆盖1Gb大小，所以映射10Gb，需要10个中级页表，所以占用40Kb；根页表，则需要一张，也就是4Kb，所以一共需要20.044Mb

2.5 如何实现 Lazy 页面分配，缺页时怎样处理？

执行mmap时可以只在MemorySet中记录合法的虚拟取件，权限和数据来源，暂时不分配物理页，也不建立有效的叶子pte，等到需要使用到物理地址的时候，也就是发生缺页异常，内核会根据stval找到对应的虚拟区域，确认权限符合，然后再分配物理页并清零，写入pte，执行sfence.vma，最后重新执行异常指令。

2.6 页面被换出到磁盘后，PTE 如何表示？

页面换出后可以清除pte的v，使得硬件访问该页时产生缺页异常。操作系统还需要再rsw，pte的软件可用位或者独立的辅助数据结构中记录该页面已换出以及交换区位置。缺页处理程序读取这些信息，将页面换入物理内存后再恢复有效pte。

问题3

3.1 单页表情况下如何更换页表？

发生进程切换时，需要进行换页，就是把satp切换到下一个进程的页表中，并更新TLB

3.2 单页表中如何阻止用户态访问内核页面？

处理器处于内核态的时候，内核页面的pte不设置u，只是会根据用途配置r，w，x；而在u态的时候会检查u，因此即使用户程序自动内核虚拟地址，也不能访问这些界面。

3.3单页表有什么优势？

u态和s态切换时，不需要频繁写入satp和刷新tlb，因此发生trap时，开销会低；内核也可以在进行权限检查后直接访问当前进程的用户地址，不需要额外切换页表或者建立临时映射。

3.4双页表何时更换页表？如果自己实现单页表，会怎样选择？

双页表中，用户程序运行时使用用户页表，发生trap后切换到内核页表；任务切换后，还要选择下一个任务的用户页表。

如果实现单页表操作系统，我会在进程切换时更换页表，但在同一个进程的用户态和内核态切换的时候用同一个页表，通过权限位来进行区分隔离。

荣誉准则说明

1.在完成本实验及此前学习过程中，我曾使用 OpenAI辅助分析测试输出、梳理系统调用路径，并讨论 TimeVal 跨页复制、PTE 权限检查、系统调用计数、mmap/munmap、MemorySet、MapArea 和物理页生命周期等概念。

2.我参考了 rCore Tutorial Guide 第四章练习、rCore-Tutorial-Book-v3、RISC-V SV39 相关说明、项目源码及官方测试程序，用于确认接口语义、页表格式和预期行为。正式提交前，我会在受这些交流或资料影响的相关代码位置注明来源。

3.除上述已经披露的交流和资料外，其余实验代码与文档由我独立完成。我理解外部帮助可能降低实验难度，并可能影响评分。

4.我没有复制或变换他人的实验代码，也不会向他人公开自己的实验实现；提交内容无意破坏或妨碍评测及其他计算机系统，并接受课程纪律约束。